🤖 NDT-Based Robot Localization from LiDAR Scans

This project implements a **Normal Distributions Transform (NDT)** localization system in Python. It allows a robot to determine its precise position on a 2D map by matching real-time LiDAR sensor data against a statistically modeled environment.

🚀 Project Overview

Localization is a fundamental challenge in robotics. While robots use wheel encoders (odometry) to estimate movement, these sensors accumulate error over time due to slippage. This project solves that by "snapping" LiDAR scans onto a pre-existing map using NDT.

Instead of matching individual pixels, NDT divides the map into a grid and calculates the **Mean** and **Covariance** of obstacles in each cell. This creates a smooth "probability surface" that a mathematical optimizer can use to find the best fit for the robot's sensor data.

---

1. Calculating the Center and Shape

First, the map is broken down into small square cells. For every cell that contains a wall, the code calculates two things:

- **The Mean (The Center):** This is the average position of all the wall pixels in that specific cell. It tells the robot, "The heart of the obstacle is exactly here."
- **The Covariance (The Shape):** This describes how the wall pixels are spread out. It tells the robot if the wall is a flat horizontal line, a vertical pillar, or a diagonal corner.

2. The Probability Score

When the robot takes a LiDAR scan, it gets a set of points. The "Score" is a calculation that asks: *"How likely is it that this laser point belongs to this wall?"*

- If a laser point lands exactly on the **Mean (Center)** of a wall cell, it gets a perfect score.

- The further a point is from that center, the lower the score becomes, following a bell-curve (Gaussian) drop-off.
- The **Covariance (Shape)** tells the math how fast that score should drop. For example, if a wall is very thin, the score drops quickly as you move away from the center.

3. Moving from Local to Global Space

The robot "sees" distances relative to itself (e.g., "there is a wall 2 meters in front of me"). To use the map, the code must translate these into map coordinates (e.g., "there is a wall at Pixel X, Pixel Y").

- The code uses the robot's current **heading (angle)** to rotate the laser points.
- It then uses the robot's **current position** to shift those points onto the map.

4. The Hill-Climbing Optimization

This is the "Correction" phase. The robot knows its wheel data (Odometry) is slightly wrong. To fix it, it performs a search:

1. **Check the Current Score:** It calculates how well its current "guess" aligns with the map.
2. **Test the Neighborhood:** It virtually "nudges" its position slightly (a few centimeters left, right, or a tiny rotation).
3. **Compare:** It checks the score for every nudge.
4. **Move:** If a nudge results in a higher score (better alignment), it moves its official position to that new spot.
5. **Repeat:** It does this over and over until it finds the "peak" of the hill—the absolute best mathematical alignment between the laser and the map.